

EST. 2025

SECURE DEV LABS

SOFTWARE AGENCY

Crafting Secure Digital Experiences

Ethical Hacking Internship

Week 3 - Web Vulnerability Validation

Class Focus:

- Move from mapping into controlled testing of common web application weaknesses

Topics Covered:

- SQL injection basics in a lab
- Types of SQLi
- XSS basics in a lab
- Types of XSS
- Cookie Stealing by XSS

Weekly Task:

- **SQLi:**
 - By using the Vulnerable Web Application find out the input fields where the sql can be performed – Provide Screenshot.
 - Mention what Vulnerable Web Application you are using?
 - What is SQL injection (SQLi), and is it still considered a major security threat today?
 - Suppose a login query is: `SELECT * FROM users WHERE username='input' AND password='input' ;` What payload could you enter to bypass authentication using a Boolean condition?
 - A query returns 2 columns: `name` and `email`. What must be true about your `UNION SELECT` statement for the injection to work?
 - You inject `OR 1=1` and the page shows all users, but `OR 1=2` shows none. What type of SQL injection technique is this and why?

- **Types of SQLI:**

- **Practical:**

- Manually perform an error-based SQL injection on the given endpoint and provide a screenshot of the resulting database error.
 - Then manually perform a Boolean-based SQL injection on the endpoint and provide a screenshot showing the data returned by the payload.
 - Finally, automate the process using sqlmap: first enumerate the database names, then list the tables within the target database, and finally extract all records from the selected table – Provide screenshots of the existing databases, then the tables within those databases, and finally all records retrieved from each table.

- **Theory:**

- What is the difference between Boolean-based SQL injection and Error-based SQL injection in terms of how information is extracted from the database?
 - How does a UNION-based SQL injection work, and what are the two main requirements for it to succeed? T

- **Conceptual Scenario Questions:**

- A web application shows different responses when you use `OR 1=1` and `OR 1=2`, but no error messages are displayed. What type of SQL injection is this, and how does it help an attacker extract data?
 - You notice that injecting ' `UNION SELECT null, null--` changes the output of a page and displays additional data. What does this indicate about the SQL query structure?
 - When testing an endpoint, you receive detailed database error messages after injecting a malformed query. What type of SQL injection is this, and how can the error information be useful to an attacker?

- **Cross Site Scripting (XSS):**

- By using the Vulnerable Web Application find out the input fields where the xss can be performed – Provide Screenshot.
- Mention what Vulnerable Web Application you are using?
- What is Cross-Site Scripting (XSS) and why is it considered a security risk?
- How can user input lead to the execution of malicious scripts in a web application?
- What are two basic methods developers use to prevent XSS vulnerabilities?

- **Types of XSS:**

- **Practical:**
 - Perform a reflected XSS attack on the input field to display the user's cookie in an alert popup. Use two different payloads—one with a `<script>` tag and another using an SVG element with an event handler. Provide both payloads separately along with screenshots of the resulting popup messages showing the user's cookie.
 - Perform a stored XSS attack on the comment/review field by injecting payloads that persist and display the user's cookie in a popup on page refresh. Use both `<script>` and `<img>` tag-based payloads, and provide the payloads along with screenshots of the resulting popups showing the cookie.
 - Finally, perform either a reflected or stored XSS attack (your choice) on the target input field. Apply payload obfuscation techniques to attempt to bypass high-security filters. Provide the payload used, the tags/techniques applied, and a screenshot of the resulting popup message displaying the cookie.
- **Theory:**
 - Briefly describe how you used the `<img>` tag with event handlers (such as `onerror`) as an alternative execution method when `<script>` execution is restricted, and how persistence makes stored XSS more impactful compared to reflected XSS.
 - Describe how you constructed the stored XSS payloads for both `<script>` and `<img>` tags in the comment/review field. Explain how the payload persists in the application database and triggers execution on page reload or when other users access the page.
 - Explain the approach you used to perform payload obfuscation in either a reflected or stored XSS attack. Describe the techniques applied to bypass security filters, such as encoding, breaking keywords, or using alternative

HTML elements and event handlers instead of standard `<script>` tags. Briefly explain why obfuscation is useful in real-world scenarios where input validation and security mechanisms attempt to block direct malicious scripts, and how different tag/event combinations can help achieve successful execution.

- **Mitigations for SQLi & XSS:**

- What is the most effective way to prevent SQL injection attacks in a web application?
- How do parameterized queries (prepared statements) help in mitigating SQL injection vulnerabilities?
- Why is input validation alone not considered a fully reliable solution for preventing SQL injection?
- What are the most effective methods to prevent Cross-Site Scripting (XSS) attacks in web applications?
- How does output encoding help in mitigating XSS vulnerabilities?
- Why is relying only on input validation not enough to fully prevent XSS attacks?